```
t = i / HEIGHT
            r = int(5 + t * 8)
            g = int(8 + t * 14)
            b = int(14 + t * 40)
            pygame.draw.line(surface, (r, g, b), (0, i), (WIDTH, i))
        if success_complete_timer % 3 == 0:
            rng_s = random.Random(success_complete_timer // 3)
            for _ in range(80):
                sx = rng_s.randint(0, WIDTH - 1)
                sy = rng_s.randint(0, HEIGHT - 1)
                surface.set_at((sx, sy), (14, 16, 24))

        # Card
        card_w, card_h = 540, 400
        card_x = WIDTH // 2 - card_w // 2
        card_y = HEIGHT // 2 - card_h // 2 - 10
        shadow = pygame.Surface((card_w + 16, card_h + 16), pygame.SRCALPHA)
        shadow.fill((0, 0, 0, 90))
        surface.blit(shadow, (card_x + 8, card_y + 8))
        card = pygame.Surface((card_w, card_h), pygame.SRCALPHA)
        card.fill((8, 14, 30, 235))
        pygame.draw.rect(card, (70, 140, 95, 160), (0, 0, card_w, card_h), 1)
        pygame.draw.rect(card, (50, 180, 100), (0, 0, 4, card_h))
        surface.blit(card, (card_x, card_y))

        def _bl_text(surf, x, y, alpha):
            surf.set_alpha(min(255, alpha))
            surface.blit(surf, (x, y))

        # Header area
        pygame.draw.rect(surface, (8, 50, 34, 200),
                         (card_x + 4, card_y, card_w - 4, 44))
        pygame.draw.line(surface, (35, 80, 55),
                         (card_x + 16, card_y + 44),
                         (card_x + card_w - 16, card_y + 44), 1)
        eyebrow = font_terminal_small.render(
            "SMART ENTRY — AFTER ACTION REPORT", True, (60, 160, 100))
        _bl_text(eyebrow,
                 card_x + card_w // 2 - eyebrow.get_width() // 2,
                 card_y + 8, success_complete_fade)
        classif = font_terminal_small.render("CLASSIFIED", True, (180, 40, 40))
        _bl_text(classif, card_x + card_w - classif.get_width() - 16,
                 card_y + 8, success_complete_fade)
        title_font = pygame.font.SysFont("Consolas", 24, bold=True)
        title_surf = title_font.render("MISSION COMPLETE", True, (100, 220, 140))
        _bl_text(title_surf,
                 card_x + card_w // 2 - title_surf.get_width() // 2,
                 card_y + 56, success_complete_fade)
        op_name = getattr(gs, "active_mission_operation", "[illegible]")
        op = font_terminal.render(op_name, True, (140, 170, 140))
        _bl_text(op,
                 card_x + card_w // 2 - op.get_width() // 2,
                 card_y + 90, success_complete_fade)
        outcome_line = getattr(gs, "active_mission_outcome", "Target escaped  ·  Network revealed  ·  Questions remain")
        out = font_terminal_small.render(outcome_line, True, (160, 140, 90))
        _bl_text(out,
                 card_x + card_w // 2 - out.get_width() // 2,
                 card_y + 116, success_complete_fade)
        pygame.draw.line(surface, (30, 50, 70),
                         (card_x + 30, card_y + 140),
                         (card_x + card_w - 30, card_y + 140), 1)

        # Stats grid
        if success_complete_timer > 30:
            stats_fade = min(255, (success_complete_timer - 30) * 7)
            report = _mission_report_data(success=True)
            stats = [
                ("KILLS", str(report["kills"])),
                ("SHOTS FIRED", str(report["shots_fired"])),
                ("ACCURACY", f"{report['accuracy']}%"),
                ("TIME TAKEN", report["time_str"]),
                ("CIVILIANS HARMED", str(report["civilians_harmed"])),
                ("INTEL FOUND", "YES" if report["intel_found"] else "NO"),
                ("[illegible]", str(report["[illegible]"])),
                ("RANK", report["rank"]),
            ]
            stat_x = card_x + 30
            stat_y = card_y + 158
            for i, (label, value) in enumerate(stats):
                col = i % 2
                row = i // 2
                sx = stat_x + col * 250
                sy = stat_y + row * 46
                lbl = font_terminal_small.render(label, True, (70, 90, 110))
                lbl.set_alpha(stats_fade)
                surface.blit(lbl, (sx, sy))
                val_font = pygame.font.SysFont("Consolas", 18, bold=True)
                val_col = (255, 200, 60) if label == "RANK" else (200, 230, 200)
                val = val_font.render(value, True, val_col)
                val.set_alpha(stats_fade)
                surface.blit(val, (sx, sy + 14))
                line_alpha = min(stats_fade, 60)
                ul = pygame.Surface((200, 1), pygame.SRCALPHA)
                ul.fill((70, 100, 80, line_alpha))
                surface.blit(ul, (sx, sy + 38))

        pygame.draw.line(surface, (30, 50, 70),
                         (card_x + 30, card_y + 340),
                         (card_x + card_w - 30, card_y + 340), 1)

        # Handler quote
        if success_complete_timer > 70:
            q_fade = min(255, (success_complete_timer - 70) * 5)
            attr = font_terminal_small.render(
                getattr(gs, "active_mission_quote_attr", "Handler, after a long pause:"), True, (90, 100, 70))
            attr.set_alpha(q_fade)
            surface.blit(attr, (card_x + 30, card_y + 356))
            quote = font_dialogue.render(
                getattr(gs, "active_mission_quote", "\"Bring it straight to me. Don't read it.\""),
                True, C_WHITE)
            quote.set_alpha(q_fade)
            surface.blit(quote,
                         (card_x + card_w // 2 - quote.get_width() // 2,
                          card_y + 374))

        # Bottom bar
        pygame.draw.line(surface, (25, 40, 55),
                         (card_x + 16, card_y + card_h - 30),
                         (card_x + card_w - 16, card_y + card_h - 30), 1)
        footer = font_terminal_small.render(
            "[illegible]", True, (30, 50, 40))
        _bl_text(footer,
                 card_x + card_w // 2 - footer.get_width() // 2,
                 card_y + card_h - 24, success_complete_fade)

        # Continue prompt
        if success_complete_timer > 140 and (pygame.time.get_ticks() // 400) % 2 == 0:
            next_prompt = "PRESS SPACE TO START MISSION 2" if getattr(gs, "active_mission", "MISSION1") == "MISSION1" else "PRESS SPACE TO RETURN TO BASE"
            prompt = font_hdr.render(next_prompt, True, C_GREEN)
            prompt.set_alpha(success_complete_fade)
            surface.blit(prompt,
                         (WIDTH // 2 - prompt.get_width() // 2,
                          card_y + card_h + 20))

# ==========================================
# [illegible] REGISTRATION
# ==========================================

for _state_cls in (
    _SplashState,
    _MenuState, _SettingsMenuState, [illegible], [illegible],
    [illegible], [illegible], _GameState, _PauseState,
    [illegible], [illegible], [illegible],
    _DeathState, _MissionCompleteState,
):
    gsm.add(_state_cls())

gsm.transition("[illegible]")
_apply_dev_args()

# ------------------------------------------
# DEV TOOLS — Level Editor & Console
# ------------------------------------------
level_editor = LevelEditor()
console = Console()

def _update_editor_map():
    mission = getattr(gs, "active_mission", "MISSION1")
    try:
        if mission == "MISSION1":
            import mission1
            level_editor.set_map(mission1.mission1_map, map_name="mission1")
        elif mission == "MISSION2":
            import mission2
            level_editor.set_map(mission2.MISSION2_MAP, map_name="mission2")
        elif mission == "MISSION3":
            import mission3
            level_editor.set_map(mission3.MISSION3_MAP, map_name="mission3")
    except (ImportError, AttributeError):
        pass

# ------------------------------------------
# Apply saved display mode on startup
# ------------------------------------------
try:
    apply_display_mode(display_mode, menu.SETTINGS_RESOLUTIONS[res_idx])
except Exception as _e:
    print(f"Startup display apply failed (non-fatal): {_e}")

# ==========================================
# MAIN LOOP — CLEAN FSM DISPATCH
# ==========================================

MAX_DT = 0.05
running = True

while running:
    dt = min(clock.tick(60) / 1000.0, MAX_DT)
    state.dt = dt
    mouse_x, mouse_y = get_scaled_mouse()
    keys = pygame.key.get_pressed()

    if hasattr(audio, "update"):
        audio.update()

    # -- Mouse visibility --
    current_name = gsm.current_name
    mouse_visible = current_name not in ("GAME", "INTRO")
    if current_name == "GAME":
        if level_editor.active:
            mouse_visible = True
        else:
            try:
                import upload_ui
                if upload_ui.is_upload_ui_active():
                    mouse_visible = False
            except ImportError:
                pass
    pygame.mouse.set_visible(mouse_visible)

    # -- Event dispatch --
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
            continue

        # Global: mute on focus loss
        if event.type == pygame.ACTIVEEVENT:
            try:
                from user_settings import SETTINGS as _s
                if _s.mute_on_focus_loss:
                    if event.gain == 0:
                        audio.mute_all()
                    else:
                        audio.unmute_all()
            except Exception:
                pass

        # Global: F11 fullscreen toggle
        if event.type == pygame.KEYDOWN and event.key == pygame.K_F11:
            new_mode = 1 if display_mode == 0 else 0
            apply_display_mode(new_mode, menu.SETTINGS_RESOLUTIONS[res_idx])
            _persist_settings()
            continue

        # F10 — Level Editor toggle (gameplay only)
        if event.type == pygame.KEYDOWN and event.key == pygame.K_F10:
            if gsm.current_name == "GAME":
                if level_editor.map_data is None:
                    _update_editor_map()
                level_editor.toggle()
            continue

        # `~` — Developer Console toggle (any state)
        if event.type == pygame.KEYDOWN and event.key == pygame.K_BACKQUOTE:
            console.toggle()
            continue

        # Console consumes all events while active
        if console.active:
            console.handle_event(event)
            continue

        # Level Editor processes events alongside game
        if level_editor.active:
            level_editor.handle_event(event)

        # State-based event handling
        if not gsm.transitioning:
            gsm.current.handle_event(event, mouse_x, mouse_y, keys)

    # -- Update dispatch --
    # NOTE: `keys` is read as a module-level global. It's set each frame
    # at the top of the loop. This works because Python's while-loop
    # doesn't create a new scope. If the loop is ever moved into a
    # function, `keys` must be passed as a parameter to update().
    if not gsm.transitioning:
        gsm.current.update(dt)

    # -- Ambience dispatch --
    gsm.current.set_ambience()
    # Stream menu music on menu-family states
    music_state = gsm.next_state if gsm.transitioning and gsm.fade_speed > 0 else current_name
    _set_menu_music(music_state in MENU_MUSIC_STATES)

    # -- Render dispatch --
    gsm.current.render(screen)

    # -- Dev tool overlays --
    level_editor.draw(screen)
    console.draw(screen)

    # -- FSM transition overlay --
    gsm.update_transition()
    gsm.draw_transition(screen)

    # -- Global post-processing --
    _apply_postprocess()

    _present_frame()
    pygame.display.flip()

pygame.quit()
sys.exit()
```